

Cómo evitar código duplicado en PL/SQL — 3 soluciones con ejemplos

Tienes razón en querer evitar duplicar código. En entornos empresariales esto es **obligatorio**, porque si cambia una regla de negocio, tienes que actualizarla en un solo lugar y no en 10 procedimientos.

Te doy 3 soluciones concretas con ejemplos reales de tu caso (puntos de venta y bitácora), y al final te digo cuál usar en cada situación.

El problema típico

Imagina que tienes esto repetido en 5 procedimientos distintos:

sql

```
-- Repetido en PR_BLOQUEAR_PV
SELECT COUNT(*), MAX(TRUNC(SYSDATE - f.fecha_vencimiento)), SUM(f.saldo_pendiente)
  INTO v_cant, v_dias, v_saldo
  FROM cliente c
  JOIN servicio s ON s.id_cliente = c.id_cliente
  JOIN factura f ON f.id_servicio = s.id_servicio
 WHERE c.id_punto_venta = p_id_pv
  AND f.estado IN ('VENCIDA', 'EN_MORA')
  AND f.saldo_pendiente > 0;

-- Y otra vez en PR_DESBLOQUEAR_PV
SELECT COUNT(*), MAX(TRUNC(SYSDATE - f.fecha_vencimiento)), SUM(f.saldo_pendiente)
  INTO v_cant, v_dias, v_saldo
  FROM cliente c
  JOIN servicio s ON s.id_cliente = c.id_cliente
  JOIN factura f ON f.id_servicio = s.id_servicio
 WHERE c.id_punto_venta = p_id_pv
  AND f.estado IN ('VENCIDA', 'EN_MORA')
  AND f.saldo_pendiente > 0;
```

Y también el INSERT en la bitácora repetido en todos lados:

```
sql

INSERT INTO historial_bloqueo_pv (...)
VALUES (seq_historial.NEXTVAL, p_id_pv, 'ACTIVO', 'INACTIVO', p_motivo, ...);
```

Vamos a eliminar ambos casos con 3 técnicas.

A Solución 1: Vista (para consultas repetidas)

Cuándo usarla: Cuando repites la **misma consulta** en varios lugares.

Creas una vista que encapsule los JOINS:

```
sql

CREATE OR REPLACE VIEW V_PV_MORA_RESUMEN AS
SELECT c.id_punto_venta,
       COUNT(DISTINCT c.id_cliente)           AS clientes_mora,
       MAX(TRUNC(SYSDATE - f.fecha_vencimiento)) AS dias_mora,
       SUM(f.saldo_pendiente)                 AS saldo_mora,
       MIN(c.id_cliente)                     AS primer_cliente_mora
FROM cliente c
JOIN servicio s ON s.id_cliente = c.id_cliente
JOIN factura f ON f.id_servicio = s.id_servicio
WHERE f.estado IN ('VENCIDA', 'EN_MORA')
      AND f.saldo_pendiente > 0
GROUP BY c.id_punto_venta;
```

Ahora en cualquier procedimiento que necesites estos datos:

```
sql

SELECT clientes_mora, dias_mora, saldo_mora, primer_cliente_mora
       INTO v_cant, v_dias, v_saldo, v_cliente
       FROM V_PV_MORA_RESUMEN
WHERE id_punto_venta = p_id_pv;
```

-- Si no encuentra fila, es porque el PV no tiene mora:

```
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
```

```
        v_cant := 0; v_dias := 0; v_saldo := 0; v_cliente := NULL;
```

Ventajas: Cero duplicación, la consulta se optimiza una sola vez, se puede reutilizar en Forms, reportes, etc.

Desventajas: No sirve para lógica con parámetros o lógica procedural (IF, loops).

B Solución 2: Función (para devolver un valor calculado)

Cuándo usarla: Cuando necesitas **un solo valor** calculado con parámetros.

```
sql
```

```
CREATE OR REPLACE FUNCTION FN_DIAS_MORA_PV (  
    p_id_punto_venta IN NUMBER  
) RETURN NUMBER IS  
    v_dias NUMBER;  
BEGIN  
    SELECT NVL(MAX(TRUNC(SYSDATE - f.fecha_vencimiento)), 0)  
        INTO v_dias  
        FROM cliente c  
        JOIN servicio s ON s.id_cliente = c.id_cliente  
        JOIN factura f ON f.id_servicio = s.id_servicio  
        WHERE c.id_punto_venta = p_id_punto_venta  
        AND f.estado IN ('VENCIDA', 'EN_MORA')  
        AND f.saldo_pendiente > 0;  
  
    RETURN v_dias;  
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        RETURN 0;  
END FN_DIAS_MORA_PV;  
/
```

Uso en cualquier procedimiento:

```
sql
```

```
IF FN_DIAS_MORA_PV(p_id_pv) > 30 THEN
    -- lógica para mora grave
END IF;
```

Ventajas: Reutilizable, se puede usar en SELECT, WHERE, etc.

Desventajas: Solo devuelve **un valor**. Si necesitas varios, se complica.

📌 Solución 3: Procedimiento centralizador (LA MEJOR para lógica compleja)

Cuándo usarla: Cuando tienes **lógica completa** repetida (varios pasos, INSERTs, validaciones, etc.).

Ejemplo 1: Procedimiento que graba en la bitácora

```
sql

-- En el paquete PKG_PROCESOS_BLOQUEOS
PROCEDURE PR_GRABAR_BITACORA (
    p_id_punto_venta    IN NUMBER,
    p_estado_anterior   IN VARCHAR2,
    p_estado_nuevo      IN VARCHAR2,
    p_motivo             IN VARCHAR2,
    p_usuario_app       IN VARCHAR2,
    p_origen             IN VARCHAR2 DEFAULT 'MANUAL',
    p_id_cliente        IN NUMBER   DEFAULT NULL,
    p_dias_mora         IN NUMBER   DEFAULT NULL
) IS
BEGIN
    INSERT INTO historial_bloqueo_pv (
        id_historial, id_punto_venta, estado_anterior, estado_nuevo,
        motivo, usuario_bd, usuario_app, origen, id_cliente,
        dias_mora, fecha_accion
    ) VALUES (
        seq_historial_bloqueo.NEXTVAL, p_id_punto_venta,
        p_estado_anterior, p_estado_nuevo,
```

```

        p_motivo, USER, p_usuario_app, p_origen, p_id_cliente,
        p_dias_mora, SYSTIMESTAMP
    );
END PR_GRABAR_BITACORA;

```

Uso en cualquier parte:

```

sql

-- En PR_BLOQUEAR_PV
UPDATE punto_venta SET estado = 'INACTIVO' WHERE id_punto_venta = p_id_pv;

PKG_PROCESOS_BLOQUEOS.PR_GRABAR_BITACORA(
    p_id_punto_venta => p_id_pv,
    p_estado_anterior => 'ACTIVO',
    p_estado_nuevo   => 'INACTIVO',
    p_motivo          => 'Bloqueo por mora',
    p_usuario_app     => p_usuario_app,
    p_origen          => 'AUTOMATICO',
    p_dias_mora       => FN_DIAS_MORA_PV(p_id_pv)
);

-- En PR_DESBLOQUEAR_PV
UPDATE punto_venta SET estado = 'ACTIVO' WHERE id_punto_venta = p_id_pv;

PKG_PROCESOS_BLOQUEOS.PR_GRABAR_BITACORA(
    p_id_punto_venta => p_id_pv,
    p_estado_anterior => 'INACTIVO',
    p_estado_nuevo   => 'ACTIVO',
    p_motivo          => 'Desbloqueo por arreglo',
    p_usuario_app     => p_usuario_app
);

-- En el trigger de auditoría
PKG_PROCESOS_BLOQUEOS.PR_GRABAR_BITACORA(
    p_id_punto_venta => :NEW.id_punto_venta,
    p_estado_anterior => :OLD.estado,
    p_estado_nuevo   => :NEW.estado,
    p_motivo          => 'Cambio detectado por trigger',
    p_usuario_app     => 'TRIGGER',
    p_origen          => 'AUTOMATICO'
);

```

Ejemplo 2: Procedimiento que devuelve el resumen de mora

sql

```
-- En el paquete PKG_PROCESOS_BLOQUEOS
PROCEDURE PR_OBTENER_RESUMEN_MORA (
    p_id_punto_venta      IN  NUMBER,
    p_clientes_mora_out   OUT NUMBER,
    p_dias_mora_out       OUT NUMBER,
    p_saldo_mora_out      OUT NUMBER,
    p_id_cliente_out      OUT NUMBER
) IS
BEGIN
    SELECT NVL(COUNT(DISTINCT c.id_cliente), 0),
           NVL(MAX(TRUNC(SYSDATE - f.fecha_vencimiento)), 0),
           NVL(SUM(f.saldo_pendiente), 0),
           MIN(c.id_cliente)
    INTO p_clientes_mora_out,
         p_dias_mora_out,
         p_saldo_mora_out,
         p_id_cliente_out
    FROM cliente c
    JOIN servicio s ON s.id_cliente = c.id_cliente
    JOIN factura f ON f.id_servicio = s.id_servicio
    WHERE c.id_punto_venta = p_id_punto_venta
           AND f.estado IN ('VENCIDA', 'EN_MORA')
           AND f.saldo_pendiente > 0;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        p_clientes_mora_out := 0;
        p_dias_mora_out     := 0;
        p_saldo_mora_out    := 0;
        p_id_cliente_out    := NULL;
END PR_OBTENER_RESUMEN_MORA;
```

Uso:

sql

```
DECLARE
    v_cantidad NUMBER;
    v_dias      NUMBER;
```

```

v_saldo    NUMBER;
v_cliente  NUMBER;
BEGIN
    PKG_PROCESOS_BLOQUEOS.PR_OBTENER_RESUMEN_MORA(
        p_id_punto_venta    => 10000,
        p_clientes_mora_out => v_cantidad,
        p_dias_mora_out     => v_dias,
        p_saldo_mora_out    => v_saldo,
        p_id_cliente_out    => v_cliente
    );

    DBMS_OUTPUT.PUT_LINE('Clientes en mora: ' || v_cantidad);
    DBMS_OUTPUT.PUT_LINE('Días de mora: '    || v_dias);
    DBMS_OUTPUT.PUT_LINE('Saldo: '          || v_saldo);
END;
/

```

Ejemplo 3: Procedimiento que valida si un PV se puede bloquear

```

sql

-- En el paquete PKG_PROCESOS_BLOQUEOS
PROCEDURE PR_VALIDAR_BLOQUEO_PV (
    p_id_punto_venta  IN  NUMBER,
    p_puede_bloquear  OUT BOOLEAN,
    p_mensaje_out     OUT VARCHAR2
) IS
    v_estado  VARCHAR2(10);
    v_dias    NUMBER;
BEGIN
    p_puede_bloquear := FALSE;
    p_mensaje_out    := NULL;

    -- 1. Validar que exista
    BEGIN
        SELECT estado INTO v_estado
        FROM punto_venta
        WHERE id_punto_venta = p_id_punto_venta;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            p_mensaje_out := 'El punto de venta no existe.';
    END;

```

```

        RETURN;
    END;

    -- 2. Validar que no esté ya bloqueado
    IF v_estado = 'INACTIVO' THEN
        p_mensaje_out := 'El punto de venta ya está bloqueado.';
        RETURN;
    END IF;

    -- 3. Validar que tenga mora
    v_dias := FN_DIAS_MORA_PV(p_id_punto_venta);
    IF v_dias = 0 THEN
        p_mensaje_out := 'El punto de venta no tiene mora.';
        RETURN;
    END IF;

    -- Si pasa todo:
    p_puede_bloquear := TRUE;
    p_mensaje_out    := 'Validación exitosa.';
END PR_VALIDAR_BLOQUEO_PV;

```

Uso:

```

sql

DECLARE
    v_ok          BOOLEAN;
    v_mensaje     VARCHAR2(200);
BEGIN
    PKG_PROCESOS_BLOQUEOS.PR_VALIDAR_BLOQUEO_PV(
        p_id_punto_venta => 10000,
        p_puede_bloquear => v_ok,
        p_mensaje_out    => v_mensaje
    );

    IF v_ok THEN
        -- Proceder con el bloqueo
        PKG_PROCESOS_BLOQUEOS.PR_BLOQUEAR_PV(...);
    ELSE
        DBMS_OUTPUT.PUT_LINE('No se puede bloquear: ' || v_mensaje);
    END IF;
END;

```



¿Cuál es la mejor forma? — Guía de decisión

No existe UNA sola mejor forma. La respuesta correcta depende de **qué estás repitiendo**:

Lo que repites	Solución recomendada
Una consulta SELECT compleja (muchos JOINS)	Vista (Solución 1)
Un valor calculado con parámetros	Función (Solución 2)
Un INSERT/UPDATE con la misma estructura	Procedimiento (Solución 3)
Una validación con varias reglas	Procedimiento (Solución 3)
Lógica con varios pasos y condicionales	Procedimiento (Solución 3)
Un valor de una tabla por su PK	Función (Solución 2)



Mi recomendación práctica para tu caso

Usa las 3 en conjunto, así:

1. **Vista** `V_PV_MORA_RESUMEN` → para todas las consultas de mora (elimina los JOINS repetidos).
2. **Función** `FN_DIAS_MORA_PV` → para cuando solo necesitas un número (validaciones rápidas).
3. **Procedimiento** `PR_GRABAR_BITACORA` → para todos los INSERTs de historial.
4. **Procedimiento** `PR_OBTENER_RESUMEN_MORA` → para cuando necesitas los 4 valores de mora a la vez.
5. **Procedimiento** `PR_VALIDAR_BLOQUEO_PV` → para las validaciones previas.

Con esto, tus procedimientos principales quedan así de limpios:

```
sql
```

```
PROCEDURE PR_BLOQUEAR_PV (  
    p_id_pv          IN NUMBER,  
    p_usuario_app    IN VARCHAR2  
) IS
```

```

v_ok          BOOLEAN;
v_mensaje     VARCHAR2(200);
v_dias        NUMBER;

BEGIN
    -- 1. Validar
    PR_VALIDAR_BLOQUEO_PV(p_id_pv, v_ok, v_mensaje);
    IF NOT v_ok THEN
        RAISE_APPLICATION_ERROR(-20010, v_mensaje);
    END IF;

    -- 2. Bloquear
    UPDATE punto_venta SET estado = 'INACTIVO' WHERE id_punto_venta = p_id_pv;

    -- 3. Registrar bitácora
    v_dias := FN_DIAS_MORA_PV(p_id_pv);
    PR_GRABAR_BITACORA(
        p_id_punto_venta => p_id_pv,
        p_estado_anterior => 'ACTIVO',
        p_estado_nuevo   => 'INACTIVO',
        p_motivo          => 'Bloqueo por mora',
        p_usuario_app     => p_usuario_app,
        p_origen          => 'MANUAL',
        p_dias_mora       => v_dias
    );

    COMMIT;
END PR_BLOQUEAR_PV;

```

Fíjate que el procedimiento principal ya no tiene SQL complejo ni lógica duplicada. **Solo orchestra las piezas.**

Resumen final

Técnica	Cuándo usarla	Ventaja principal
Vista	Consultas SELECT complejas repetidas	Cero duplicación de JOINS
Función	Cálculo de un valor	Se usa en SELECT, IF, WHERE

Técnica	Cuándo usarla	Ventaja principal
Procedimiento	Lógica con varios pasos	Encapsula reglas de negocio

Regla de oro: Si el mismo bloque de código aparece **2 veces o más**, conviértelo en vista, función o procedimiento. **Nunca lo dejes duplicado.**

¿Quieres que te arme el paquete `PKG_PROCESOS_BLOQUEOS` completo con todos estos procedimientos y funciones juntos, listo para copiar y pegar?

WinSCP es una herramienta gratuita para Windows que funciona como un "explorador de archivos" pero para conectarte a un servidor remoto. Su función principal es **copiar archivos de forma segura entre tu computadora y un servidor**, que es exactamente lo que necesitas para mover tus archivos compilados `.fmx`.

Paso 1: Descargar e instalar WinSCP

1. Ve al sitio web oficial: <https://winscp.net/eng/download.php>.
2. Descarga el paquete de instalación "Multilanguage installation package".
3. Ejecuta el instalador. Durante la instalación, te recomiendo seleccionar la "**Explorer interface**" (interfaz de explorador), ya que es la más parecida al Explorador de Windows y te resultará más familiar.

Paso 2: Reunir los datos de conexión

Antes de abrir el programa, necesitas tener a mano la información del servidor de tu empresa. Pídesela al equipo de TI:

- **Protocolo:** Generalmente **SFTP** (el más seguro) o SCP.
- **Nombre del servidor (Host name):** Puede ser una dirección IP (ej. `192.168.1.50`) o un nombre de dominio (ej. `servidor-desarrollo.empresa.com`).
- **Puerto:** Normalmente es el **22** para SFTP, pero podría ser otro.
- **Usuario:** Tu nombre de usuario en el servidor.

•**Contraseña:** Tu contraseña de acceso al servidor. Si usan autenticación por clave, te darán un archivo de clave privada.

Paso 3: Conectarse al servidor

1. Abre WinSCP. Verás una ventana de "Login" (Inicio de sesión).
2. Rellena los campos con los datos del Paso 2:

- **File protocol (Protocolo de archivo):** Selecciona **SFTP**.
 - **Host name (Nombre del host):** Escribe la IP o el nombre del servidor.
 - **Port number (Número de puerto):** Escribe **22** (o el que te hayan indicado).
 - **User name (Nombre de usuario):** Tu usuario.
 - **Password (Contraseña):** Tu contraseña.
3. Haz clic en **Login** (Iniciar sesión).

La primera vez, WinSCP te pedirá confirmar la "huella digital" (clave) del servidor. Acepta si el equipo de TI te lo confirma.

Paso 4: Transferir tus archivos **.fmx** (Ejemplo práctico)

Una vez conectado, verás una ventana con **dos paneles**:

- **Panel izquierdo:** Tu computadora local.
- **Panel derecho:** El servidor remoto.

Tu objetivo es copiar el archivo **FOR_BLOQUEO_PV.fmx desde tu PC al servidor.**

1. **En el panel izquierdo (tu PC):** Navega hasta la carpeta donde compilaste tu formulario (por ejemplo, **C:\oracle\desarrollos**).
2. **En el panel derecho (el servidor):** Navega a la carpeta donde se guardan los formularios en el servidor. Esta es la ruta que debes preguntar a TI, a menudo se llama **FORMS_PATH** (por ejemplo, **/u01/app/oracle/forms/** o **D:\Oracle\Forms**).
3. **Arrastra el archivo:** Simplemente haz clic sobre **FOR_BLOQUEO_PV.fmx** en el panel izquierdo y **arrástralo** al panel derecho, soltándolo en la carpeta de destino.
4. WinSCP te mostrará una ventana de confirmación. Haz clic en **Copy** (Copiar) o **OK** para iniciar la transferencia.

💡 Consejos adicionales

- Sobrescribir archivos:** Si el archivo `.fmx` ya existe en el servidor, WinSCP te preguntará si deseas sobrescribirlo. Normalmente, querrás hacerlo para actualizar el formulario con tus cambios.
- Automatización:** Si necesitas transferir archivos con frecuencia, puedes usar la línea de comandos de WinSCP para crear un script que suba el archivo automáticamente. El comando básico sería similar a: `winscp.com /command "open sftp://usuario:contraseña@servidor/" "put C:\ruta\local\FOR_BLOQUEO_PV.fmx /ruta/remota/" "exit"`.
- Seguridad:** Evita guardar tu contraseña en WinSCP si trabajas en un equipo compartido. Es mejor escribirla cada vez.

Siguiendo estos pasos, podrás transferir tus archivos compilados al servidor de forma segura. Si al conectarte te aparece algún error (por ejemplo, de "host key" o de permisos), puedes consultarme de nuevo.